

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM – 535003

ANDHRA PRADESH

SKILL SHARING COMMUNITY PLATFORM

By

KAGITAPALLI DILEEP KUMAR

(Regd. No. 24VV1A1225)

Under the supervision of

MRS. MADHUMITA CHANDA

Department of Information Technology

JNTU-GV, Vizianagaram

Academic Year: 2026–2027

CONTENTS

1. Introduction…… 3
2. Project Overview and Requirements……..... 4
3. Project Architecture and Structure……. 6
4. Flutter Development Environment Setup……... 8
5. Dart Programming Fundamentals Used in the Project…….. 11
6. Creating and Configuring the Flutter Project…… 13
7. Skill Sharing Community Platform – Case Study……. 14
8. User Authentication and Profile Management…….. 16
9. Skills Management Module…….. 18
10. Learning Goals Module…….. 20
11. Skill-Based Study Matching Module…….. 22
12. Connection Request Module…….. 24
13. Exploring Flutter Widgets in the Project……... 26
14. Responsive UI Design……... 28
15. Navigation Between Screens…….. 30
16. Stateful and Stateless Widgets……... 31
17. State Management……. 32
18. Custom Widgets and Styling……. 33
19. Forms and Validation……. 35
20. REST API Integration with Java Backend……. 37
21. Database and Data Models…… 39
22. Animations and User Experience……... 41
23. Testing and Debugging……... 42
24. Application Workflow…… 44
25. Security and Data Validation…….. 49
26. Conclusion…….. 50

1.Introduction

Flutter is an open-source UI framework developed by Google for building cross-platform applications from a single codebase. Flutter uses the Dart programming language and provides a widget-based approach for designing interactive mobile applications.

The Skill Sharing Community Platform is designed to help students and learners discover people with useful skills, share their own skills, define learning goals, find suitable study partners, and send connection requests. Instead of functioning as a quiz application, the platform focuses on peer learning and skill exchange.

Advantages of Flutter

- Cross-platform development using a common codebase.
- Hot reload for quick UI development and testing.
- Large collection of customizable Material widgets.

Dart Language

Dart is the programming language used by Flutter. It is object-oriented, strongly typed, and supports asynchronous programming through Future, async, and await. These features are important in this project because the application communicates with a backend service and displays data that may arrive asynchronously.

Key Flutter Features Used in This Project

- Widget-based UI development for profile cards, skill cards, forms, lists, and navigation.
- Stateful widgets for screens whose content changes after user actions or API responses.
- Stateless widgets for reusable presentation components.
- Navigation for moving between authentication, dashboard, skill, goal, match, request, and profile screens.

2.Project Overview and Requirements

The Skill Sharing Community Platform is a mobile application that creates a structured environment for peer-to-peer learning. A user can create an account, maintain a profile, add skills they can offer, specify skills or topics they want to learn, discover compatible users, and establish connections through connection requests.

Problem Statement

Students often possess different technical and academic skills but do not have a simple platform to identify peers who can teach them a particular skill or who want to learn the same skill. The proposed platform addresses this problem by organizing user skills and learning goals and using that information to support study-partner discovery.

Main Objectives

- Provide secure user registration and login.
- Allow users to maintain and update their profile information.
- Allow users to add and manage skills they can share.
- Allow users to specify learning goals.
- Identify suitable study partners using skill and learning-goal information.
- Allow users to send and manage connection requests.
- Present information through a simple, responsive Flutter interface.
- Connect the Flutter application to a Java backend and database.

Functional Requirements

Module	Requirement
Authentication	Registration, login, logout, and validation of user credentials.
Profile	View and update user profile details.
Skills	Add, view, and associate skills with a user.

Learning Goals	Create and manage topics or skills the user wants to learn.
Study Matching	Display suitable study partners based on relevant skills and goals.
Connections	Send, receive, accept, or reject connection requests.
Backend	Exchange application data with the Java backend through APIs.
Database	Store users, skills, user-skill relationships, goals, matches, and requests.

Non-Functional Requirements

- **Usability:** the interface should be simple enough for students to understand without training.
- **Performance:** screens should load data asynchronously without freezing the UI.
- **Maintainability:** code should be separated into screens, models, services, and reusable widgets.
- **Scalability:** the structure should allow additional modules without rewriting the complete application.
- **Security:** authentication and server-side validation should protect user data and operations.

3.Project Architecture and Structure

The Flutter application follows a modular structure. The UI is separated from data models and backend communication so that each part can be maintained independently.

Recommended Project Directory

```
skill_sharing_community/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   ├── user_model.dart
│   │   ├── skill_model.dart
│   │   ├── learning_goal_model.dart
│   │   ├── study_match_model.dart
│   │   └── connection_request_model.dart
│   ├── screens/
│   │   ├── login_screen.dart
│   │   ├── register_screen.dart
│   │   ├── dashboard_screen.dart
│   │   ├── profile_screen.dart
│   │   ├── edit_profile_screen.dart
│   │   ├── skills_screen.dart
│   │   ├── add_skill_screen.dart
│   │   ├── learning_goals_screen.dart
│   │   ├── matches_screen.dart
│   │   └── connection_requests_screen.dart
│   ├── services/
│   │   ├── auth_service.dart
│   │   ├── user_service.dart
│   │   ├── skill_service.dart
│   │   ├── goal_service.dart
│   │   ├── match_service.dart
│   │   └── connection_service.dart
│   ├── widgets/
│   │   ├── custom_button.dart
│   │   ├── skill_card.dart
│   │   ├── goal_card.dart
│   │   ├── match_card.dart
│   │   └── connection_request_tile.dart
│   └── utils/
│       ├── validators.dart
│       └── constants.dart
├── android/
├── assets/
├── test/
└── pubspec.yaml
```

Description of Project Modules

Directory	Purpose
lib/	Contains the Dart source code of the Flutter application.
models/	Represents application data such as users, skills, goals, matches, and requests.
screens/	Contains complete application screens and page-level UI.
services/	Contains authentication and REST API communication logic.
widgets/	Contains reusable UI components used across multiple screens.
utils/	Contains validation rules, constants, and helper functions.
assets/	Stores application images and other static resources.
test/	Contains unit and widget tests.
pubspec.yaml	Defines Flutter dependencies, assets, and project configuration.

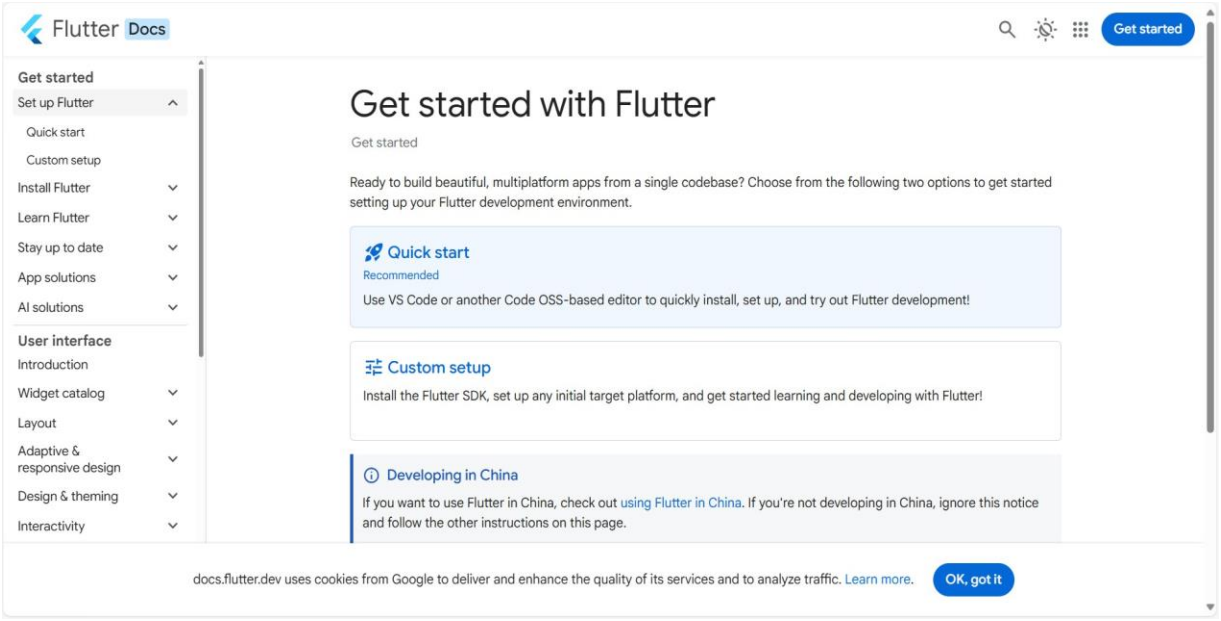
Architecture Flow

```
Flutter UI
   ↓
Screens / Reusable Widgets
   ↓
State Management
   ↓
Service Layer
   ↓
REST API
   ↓
Java Backend
   ↓
Database
```

4.Flutter Development Environment Setup

Flutter SDK Installation

1. Download the stable Flutter SDK from the official Flutter website.
2. Extract the SDK to a suitable location such as C:\src\flutter.
3. Add C:\src\flutter\bin to the Windows PATH environment variable.
4. Open a new terminal and execute flutter --version.
5. Run flutter doctor to identify missing development components.



Installation of Flutter

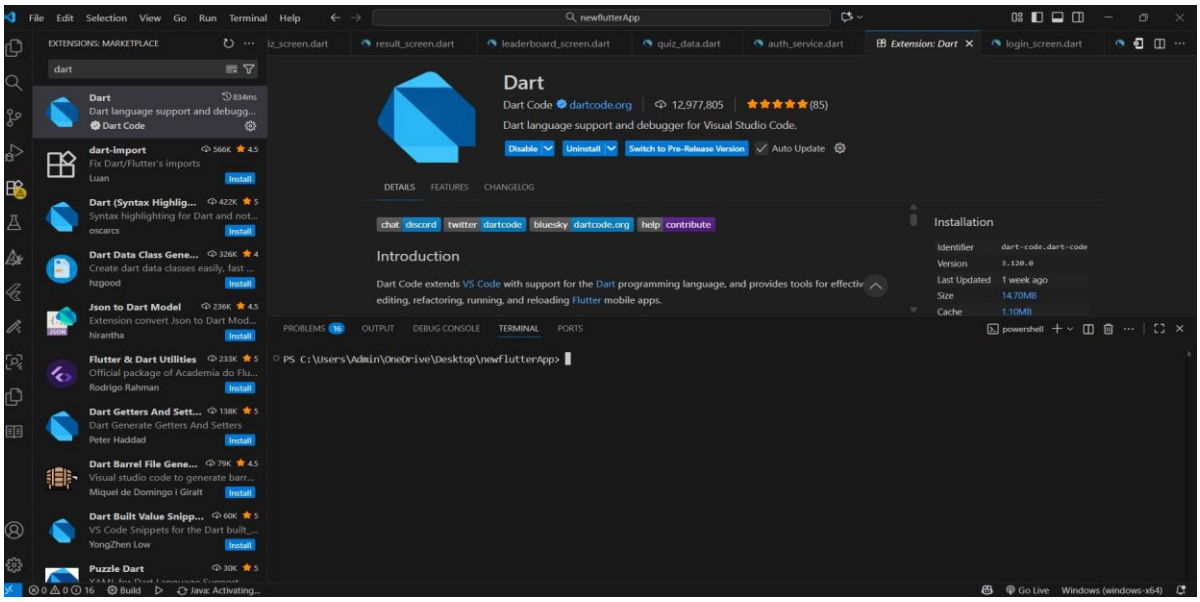
Android Toolchain Without Android Studio

The project can be developed using Visual Studio Code and the Android command-line toolchain. Android Studio is not required as the code editor for this project.

1. Install the required Android SDK command-line tools.
2. Configure the Android SDK path and required platform tools.
3. Accept Android SDK licenses using flutter doctor --android-licenses.
4. Run flutter doctor and ensure the Flutter and Android toolchain checks are available.
5. Use a connected Android device or a configured Android emulator for testing.

Visual Studio Code

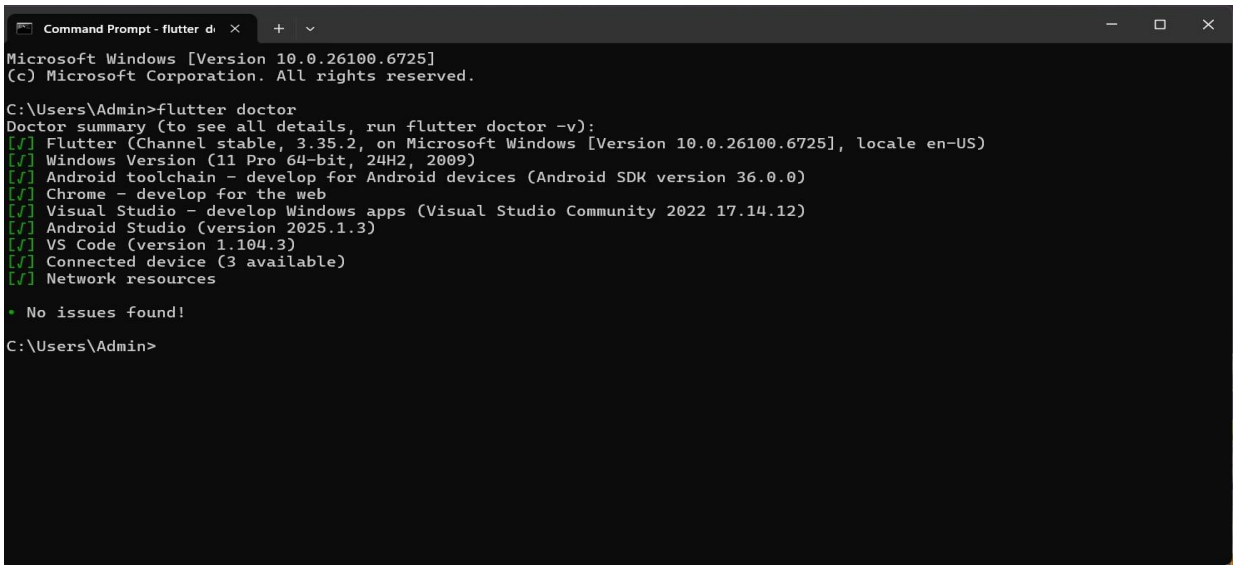
1. Install Visual Studio Code.
2. Open the Extensions panel using Ctrl + Shift + X.
3. Install the Flutter extension; the Dart extension is installed as a dependency.
4. Open the project folder in VS Code.
5. Use the integrated terminal for Flutter commands and debugging.



Installing the Dart and Flutter extensions in Visual Studio Code

Verify the Environment

```
flutter --version
dart --version
flutter doctor
flutter devices
```



Verifying the Environment

5.Dart Programming Fundamentals Used in the Project

Dart provides the programming foundation for the Flutter application. The project uses variables, classes, lists, maps, functions, conditional statements, loops, asynchronous programming, and null safety.

Variables and Data Types

```
String userName = "Dileep";
int userId = 101;
bool isConnected = false;
List<String> skills = ["Java", "Flutter", "SQL"];
Map<String, dynamic> profile = {
  "name": "Dileep",
  "department": "IT"
};
```

Conditional Statements

```
if (skills.isEmpty) {
  print("No skills added");
} else {
  print("Skills are available");
}
```

Lists

Lists are used for collections such as a user's skills, learning goals, search results, and matched users.

```
List<String> goals = [
  "Learn Flutter",
  "Improve Java",
  "Practice SQL"
];

for (final goal in goals) {
  print(goal);
}
```

Classes and Models

```
class Skill {
  final int id;
```

```
  final String name;

  Skill({
    required this.id,
    required this.name,
  });
}
```

Asynchronous Programming

REST API requests are asynchronous because the application must wait for the backend response without blocking the user interface.

```
Future<List<Skill>> fetchSkills() async {
  final response = await http.get(
    Uri.parse("$baseUrl/skills")
  );

  // Convert the response into Skill objects.
  return parseSkills(response.body);
}
```

6.Creating and Configuring the Flutter Project

Creating the Application

```
flutter create skill_sharing_community
cd skill_sharing_community
code .
flutter run
```

pubspec.yaml

The pubspec.yaml file is used to declare Flutter dependencies and project assets. The exact versions should be selected according to the project environment and compatibility requirements.

```
dependencies:
  flutter:
    sdk: flutter
  http: ^<compatible-version>
  provider: ^<compatible-version>

flutter:
  uses-material-design: true
  assets:
    - assets/images/
```

Main Application Entry Point

```
import 'package:flutter/material.dart';

void main() {
  runApp(const SkillSharingApp());
}

class SkillSharingApp extends StatelessWidget {
  const SkillSharingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Skill Sharing Community',
      home: const LoginScreen(),
    );
  }
}
```

7.Skill Sharing Community Platform – Case Study

The Skill Sharing Community Platform is the central case study for this Flutter documentation. It is designed for students who want to exchange knowledge and find peers with compatible learning interests.

Core Features

- User registration and authentication.
- User profile creation and editing.
- Skill discovery and skill management.
- Association of multiple skills with a user.
- Learning-goal creation and management.
- Study-partner matching based on skills and goals.
- Connection request creation and management.
- Responsive mobile interface.
- REST API communication with the Java backend.
- Persistent storage through the project database.

User Journey

1. A new user opens the application and creates an account.
2. The user logs in and reaches the dashboard.
3. The user completes or updates their profile.
4. The user adds skills they can offer to other learners.
5. The user adds learning goals representing topics they want to learn.
6. The application displays suitable study matches.
7. The user reviews another learner's relevant information.
8. The user sends a connection request.
9. The receiver can accept or reject the request.
10. The accepted connection becomes part of the user's learning network.

Technologies Used

Technology	Role in the Project
Flutter	Mobile application UI and navigation.
Dart	Application programming language.
Visual Studio Code	Development and debugging environment.
Java Backend	Server-side business logic and REST APIs.
Database	Persistent storage for users, skills, goals, matches, and requests.
Provider / State Management	Keeps UI synchronized with changing application data.
Git/GitHub	Source-code version control and collaboration.



Case Study of Skill Sharing Community Platform

8.User Authentication and Profile Management

Registration

The registration screen collects the information required to create a new platform account. Client-side validation prevents incomplete or invalid input before the request is sent to the backend.

- Required-field validation.
- Email format validation.
- Password validation.
- Password confirmation validation where applicable.
- Loading indicator while the registration request is processed.
- Clear error messages for failed registration.

Login

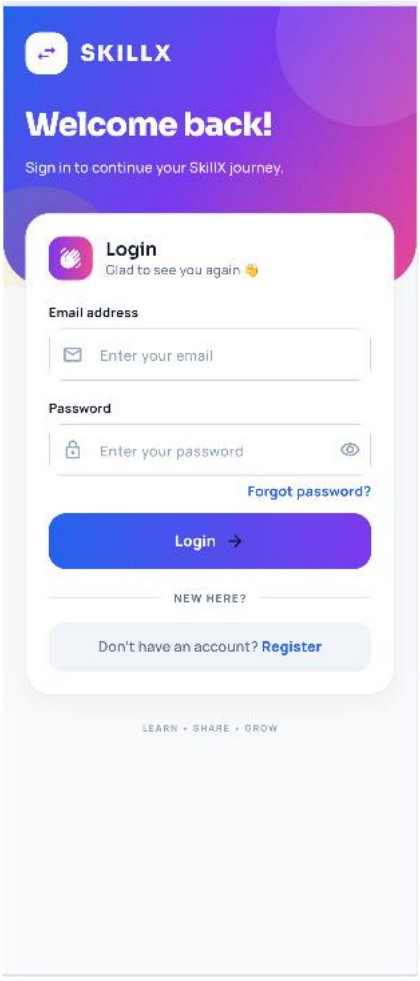
The login screen accepts the user's authentication credentials and sends them to the backend. On successful authentication, the application navigates to the authenticated dashboard.

```
Future<void> login() async {
  if (!_formKey.currentState!.validate()) return;

  setState(() => loading = true);

  try {
    await authService.login(
      emailController.text.trim(),
      passwordController.text,
    );

    if (!mounted) return;
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (_) => const DashboardScreen(),
      ),
    );
  } finally {
    if (mounted) setState(() => loading = false);
  }
}
```



Login screen

Profile Management

The profile module allows a user to view and update personal information that is relevant to the learning community. Profile information is separated from skill and goal data so that each entity can be maintained independently.

- View profile information.
- Edit profile information.
- Display associated skills.
- Display learning goals.
- Reflect updated information immediately in the UI after a successful update.

9.Skills Management Module

The Skills module is one of the main features that differentiates this project from a generic Flutter application. A skill represents knowledge or an ability that can be shared with other learners.

Skill Management Operations

- View available skills.
- Select or add skills relevant to the user.
- Associate a user with one or more skills.
- Display the user's selected skills on the profile.
- Use skill information as an input to the matching process.

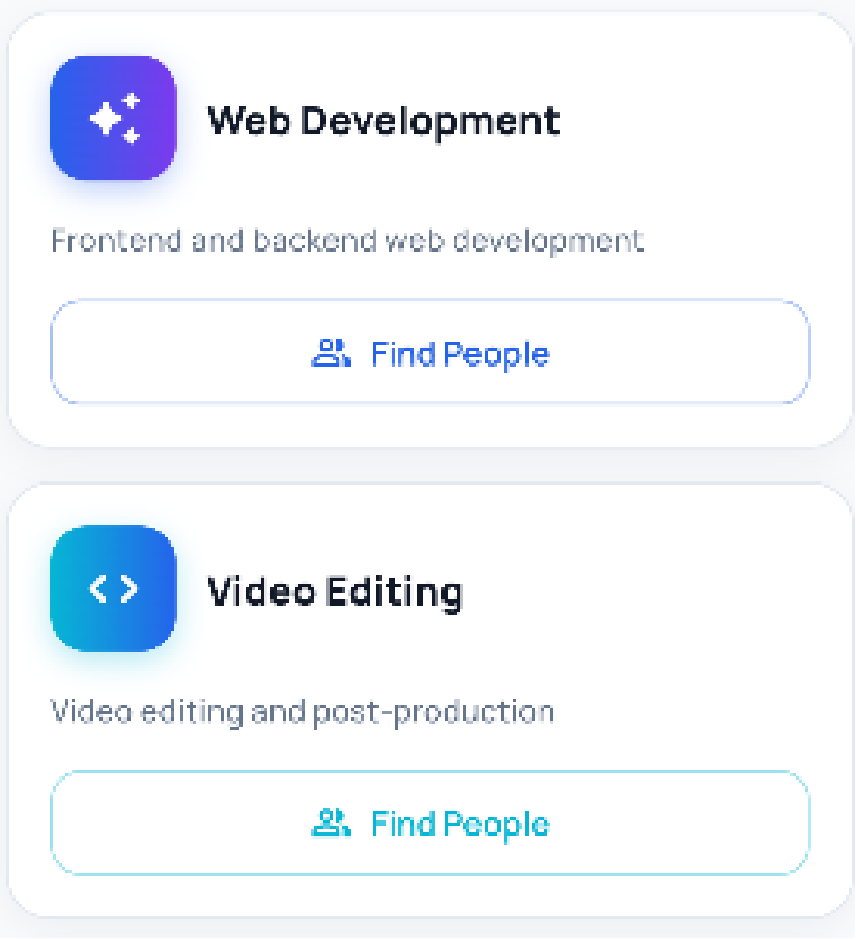
Skill Card

A SkillCard is a reusable widget that displays a skill name and, where required, an action such as add, remove, or select.

```
class SkillCard extends StatelessWidget {
  final String skillName;
  final VoidCallback? onTap;

  const SkillCard({
    super.key,
    required this.skillName,
    this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        leading: const Icon(Icons.star_outline),
        title: Text(skillName),
        onTap: onTap,
      ),
    );
  }
}
```



Skill cards

Skill-to-User Relationship

A user can have multiple skills, and the same skill can belong to multiple users. Therefore, the database uses a separate user_skills relationship table instead of storing a list directly in the users table.

10.Learning Goals Module

Learning goals represent what a user wants to learn. They are different from the skills a user can offer. This distinction is important because the matching system can compare what one person can teach with what another person wants to learn.

Learning Goal Operations

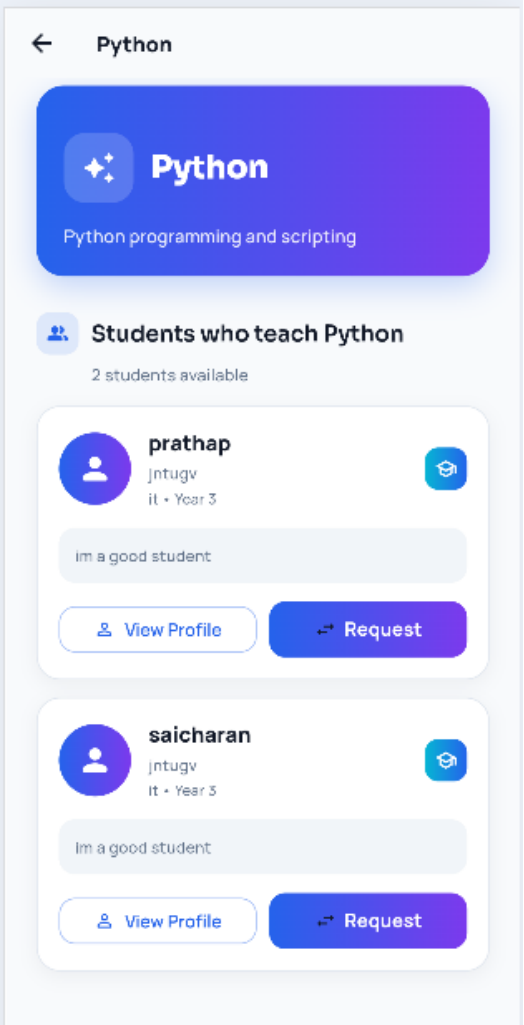
- Create a learning goal.
- View existing learning goals.
- Update a learning goal when requirements change.
- Remove an obsolete learning goal.
- Use learning goals during study-partner matching.

Goal Card

```
class GoalCard extends StatelessWidget {
  final String goal;
  final VoidCallback? onDelete;

  const GoalCard({
    super.key,
    required this.goal,
    this.onDelete,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        leading: const Icon(Icons.flag_outlined),
        title: Text(goal),
        trailing: onDelete == null
            ? null
            : IconButton(
                icon: const Icon(Icons.delete_outline),
                onPressed: onDelete,
              ),
      ),
    );
  }
}
```



Goal cards

Difference Between Skills and Goals

Item	Meaning	Example
Skill	What the user can offer or teach.	Java Programming
Learning Goal	What the user wants to learn.	Flutter Development

11.Skill-Based Study Matching Module

The Study Matching module identifies users whose skills and learning goals are relevant to each other. The purpose is to reduce the effort required for a student to find a suitable learning partner.

Matching Concept

```
User A
Skills offered: Java
Learning goal: Flutter

User B
Skills offered: Flutter
Learning goal: Java

Potential result:
User A ⇔ User B
```

Matching Flow

1. Read the current user's skills and learning goals.
2. Retrieve candidate users from the backend.
3. Compare candidate skills with the current user's goals.
4. Compare the current user's skills with candidate goals.
5. Calculate or receive a relevance score from the backend.
6. Display suitable matches as match cards.
7. Allow the user to inspect a match and send a connection request.

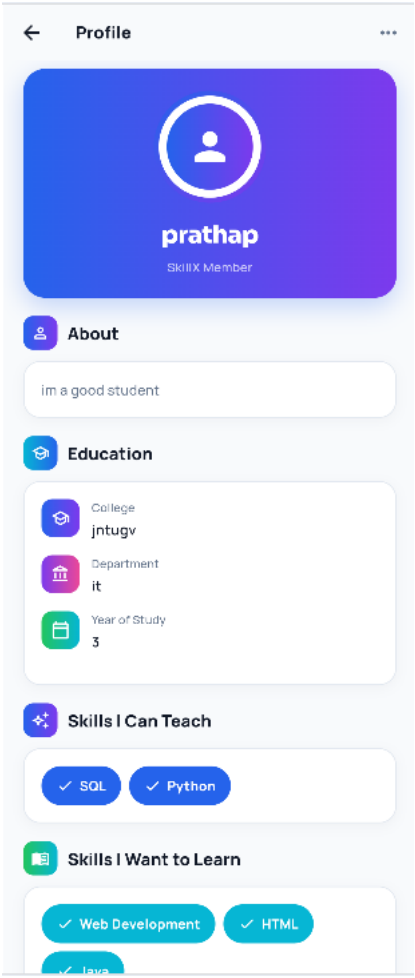
Match Card

```
class MatchCard extends StatelessWidget {
  final String userName;
  final String matchedSkill;
  final VoidCallback onConnect;

  const MatchCard({
    super.key,
    required this.userName,
    required this.matchedSkill,
```

```
    required this.onConnect,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        title: Text(userName),
        subtitle: Text("Relevant skill: $matchedSkill"),
        trailing: ElevatedButton(
          onPressed: onConnect,
          child: const Text("Connect"),
        ),
      ),
    );
  }
}
```



Match Cards

12.Connection Request Module

The connection request module provides a controlled way for users to establish a learning connection. A request is stored with a status so that the sender and receiver can see the current state of the relationship.

Request States

Status	Meaning
Pending	The request has been sent but not yet decided by the receiver.
Accepted	The receiver accepted the request and the users can be treated as connected.
Rejected	The receiver declined the request.

Connection Request Workflow

1. User A selects a suitable match.
2. User A presses the Connect action.
3. The Flutter application sends a request to the backend.
4. The backend validates the request and stores it.
5. User B views the pending request.
6. User B accepts or rejects the request.
7. The backend updates the request status.
8. The Flutter UI refreshes to display the new state.

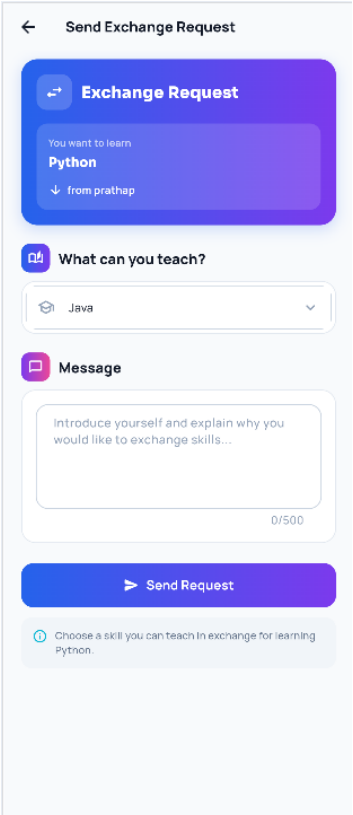
Connection Request Tile

```
class ConnectionRequestTile extends StatelessWidget {
  final String senderName;
  final VoidCallback onAccept;
  final VoidCallback onReject;

  const ConnectionRequestTile({
    super.key,
    required this.senderName,
    required this.onAccept,
```

```
    required this.onReject,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        title: Text(senderName),
        subtitle: const Text("Wants to connect with you"),
        trailing: Row(
          mainAxisSize: MainAxisSize.min,
          children: [
            IconButton(
              onPressed: onAccept,
              icon: const Icon(Icons.check),
            ),
            IconButton(
              onPressed: onReject,
              icon: const Icon(Icons.close),
            ),
          ],
        ),
      ),
    );
  }
}
```



Connections Request

13.Exploring Flutter Widgets in the Project

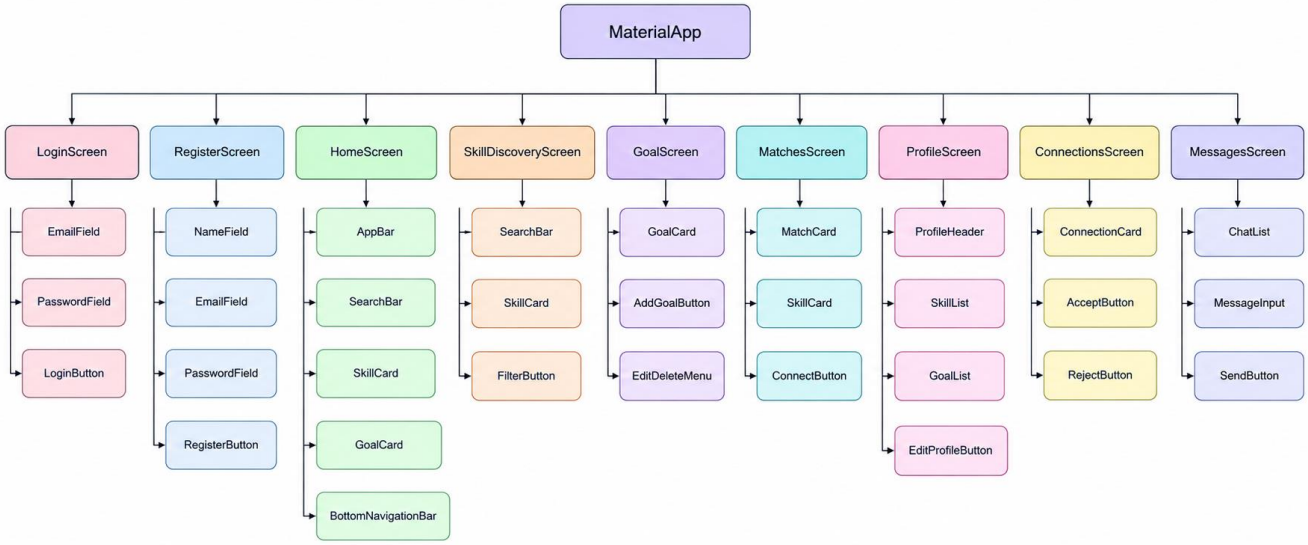
Flutter uses widgets as the fundamental building blocks of the user interface. The Skill Sharing Community Platform uses standard Flutter widgets together with custom widgets.

Important Widgets

Widget	Use in the Project
MaterialApp	Root application configuration and navigation.
Scaffold	Base structure for screens.
AppBar	Screen title and actions.
Text	Display names, descriptions, skills, goals, and messages.
TextField / TextFormField	Collect user input.
Card	Display skills, goals, matches, and requests.
ListView	Display dynamic lists.
Column	Arrange vertical content.
Row	Arrange horizontal actions.
Expanded	Use available space safely in responsive layouts.
SingleChildScrollView	Prevent overflow on smaller screens.
CircleAvatar	Display user profile images or initials.
ElevatedButton	Primary actions such as Login and Connect.

Example Widget Tree

```
MaterialApp
 └── Scaffold
      ├── AppBar
      └── Body
           └── SingleChildScrollView
                └── Column
                     ├── Profile Header
                     ├── Skills Section
                     │    └── SkillCard
                     ├── Learning Goals Section
                     │    └── GoalCard
                     └── Recommended Matches
                          └── MatchCard
```



Simplified Widget Tree of the SkillX App

14.Responsive UI Design

Responsive design ensures that the application's content remains usable across different Android screen sizes. The platform uses flexible Flutter widgets rather than fixed dimensions wherever possible.

Responsive Techniques

- SafeArea keeps important content away from system areas.
- Expanded and Flexible prevent horizontal overflow.
- ListView handles long dynamic lists.
- SingleChildScrollView allows screens to adapt to smaller heights.
- MediaQuery or LayoutBuilder can be used when a layout must change according to available width.
- Consistent padding and spacing improves readability.

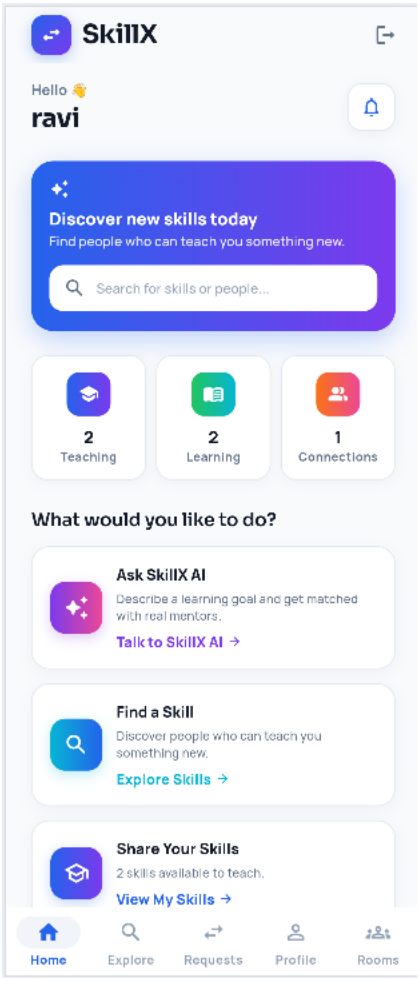
Example

```
LayoutBuilder(
  builder: (context, constraints) {
    final isWide = constraints.maxWidth > 600;

    return isWide
        ? Row(
            children: [
              Expanded(child: ProfileSection()),
              Expanded(child: SkillsSection()),
            ],
          )
        : Column(
            children: [
              ProfileSection(),
              SkillsSection(),
            ],
          );
  },
)
```

Project Application

The dashboard, skill list, learning-goal list, match list, and request list should remain accessible when the device width or height changes. The interface therefore avoids depending on fixed screen coordinates.



Home Screen

15.Navigation Between Screens

Navigation connects the independent screens of the Skill Sharing Community Platform. The authentication flow and feature modules are separated so that users reach only the screens appropriate to their current state.

Main Navigation Flow

```
Splash / Start
      ↓
   Login
      ↕
 Register
      ↓
 Dashboard
 ├── Profile
 ├── Edit Profile
 ├── Skills
 ├── Learning Goals
 ├── Study Matches
 └── Connection Requests
```

Logout

Logout should clear the authenticated session and return the user to the login screen. The navigation stack should be cleared so that the user cannot return to an authenticated screen using the back button.

```
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(
    builder: (_) => const LoginScreen(),
  ),
  (route) => false,
);
```



SkillX Logout

16.Stateful and Stateless Widgets

Stateless Widgets

A StatelessWidget is appropriate when the widget does not need to maintain mutable state internally. In this project, reusable presentation components such as SkillCard, GoalCard, MatchCard, and ConnectionRequestTile can be implemented as stateless widgets.

Stateful Widgets

A StatefulWidget is appropriate when the screen changes because of user interaction, loading operations, API results, or local state.

- Login and registration screens maintain form/loading state.
- Skill and goal screens may maintain selected or entered values.
- Match screens maintain loading and fetched-result state.
- Connection request screens update after accepting or rejecting a request.

setState Example

```
setState(() {
  loading = true;
});

try {
  await service.fetchMatches();
} finally {
  if (mounted) {
    setState(() {
      loading = false;
    });
  }
}
```

setState should be used for local UI state. For application-wide shared state, a state-management solution such as Provider can be used to expose data and notify dependent widgets when the data changes.

17.State Management

State management controls how changing application data is reflected in the UI. The project can use Provider to separate shared state from individual screen widgets.

Why State Management Is Required

- Authentication state must be available to multiple screens.
- Updated profile information may need to appear in the dashboard and profile screen.
- Skill and goal data may be reused by matching screens.
- Connection request changes should refresh the relevant UI.

Provider-Style Structure

```
ChangeNotifier
   ↓
AuthProvider
UserProvider
SkillProvider
GoalProvider
MatchProvider
ConnectionProvider
   ↓
Flutter Screens and Widgets
```

Example Provider

```
class SkillProvider extends ChangeNotifier {
  List<Skill> _skills = [];

  List<Skill> get skills => _skills;

  Future<void> loadSkills() async {
    _skills = await skillService.fetchSkills();
    notifyListeners();
  }
}
```

The exact provider classes should match the modules implemented in the final application. The main principle is that network/data logic should not be duplicated across multiple screens.

18.Custom Widgets and Styling

Reusable widgets reduce duplication and help maintain a consistent visual design. The project uses custom components for common community-platform elements.

Recommended Custom Widgets

- CustomButton – reusable action button.
- SkillCard – displays a skill.
- GoalCard – displays a learning goal.
- MatchCard – displays a recommended study partner.
- ConnectionRequestTile – displays a pending connection request.
- ProfileHeader – displays the user's name and profile information.

Styling Principles

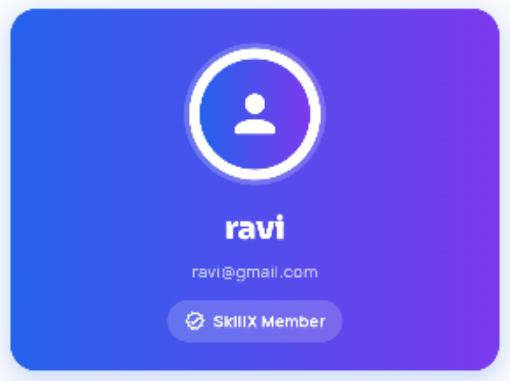
- Use a consistent color palette across all screens.
- Use rounded cards and buttons where appropriate.
- Keep primary actions visually distinct.
- Maintain consistent font sizes for headings and body text.

Reusable Button Example

```
class CustomButton extends StatelessWidget {
  final String text;
  final VoidCallback onPressed;

  const CustomButton({
    super.key,
    required this.text,
    required this.onPressed,
  });

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: onPressed,
      child: Text(text),
    );
  }
}
```



Profile Widget



Image Widget



App Icon Widget

19.Forms and Validation

Forms are used in registration, login, profile editing, adding skills, and creating learning goals. Validation prevents invalid information from being sent to the backend.

Validation Requirements

Form	Validation
Registration	Required fields, valid email, password requirements, matching confirmation.
Login	Required credentials.
Profile	Required profile fields and valid input format.
Add Skill	Skill name should not be empty.
Learning Goal	Goal description should not be empty.

Form Example

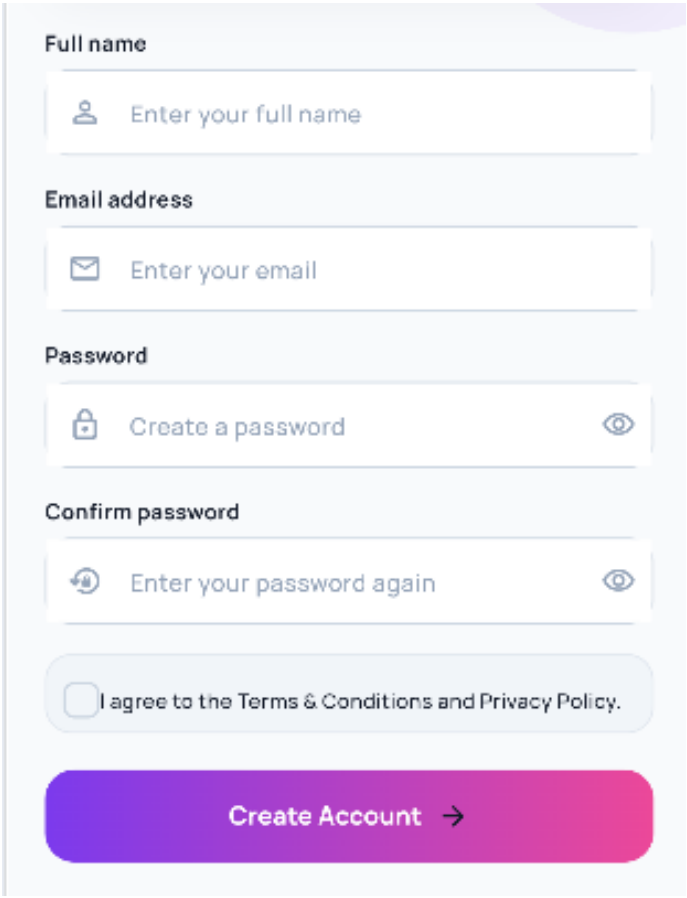
```
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    decoration: const InputDecoration(
      labelText: "Skill",
    ),
    validator: (value) {
      if (value == null || value.trim().isEmpty) {
        return "Please enter a skill";
      }
      return null;
    },
  ),
);
```

Submission

```
if (_formKey.currentState!.validate()) {
  // Send validated data to the service layer.
}
```

Client-side validation improves usability, but it must not be treated as the only security control. The backend must validate the same important constraints before modifying persistent data.



Forms Validation

20.REST API Integration with Java Backend

The Flutter application communicates with the Java backend through REST APIs. The service layer is responsible for making HTTP requests and converting responses into Dart model objects.

HTTP Operations

Method	Typical Use
GET	Retrieve users, skills, goals, matches, or requests.
POST	Create a user, skill association, goal, or connection request.
PUT/PATCH	Update profile or other editable information.
DELETE	Remove supported user-owned data such as a skill association or goal.

Example Service

```
class SkillService {
  final String baseUrl;

  SkillService(this.baseUrl);

  Future<List<Skill>> getSkills() async {
    final response = await http.get(
      Uri.parse('$baseUrl/skills'),
    );

    if (response.statusCode != 200) {
      throw Exception('Failed to load skills');
    }

    return parseSkillList(response.body);
  }
}
```

API Layering

```
Screen
  ↓
SkillService
  ↓
HTTP Request
  ↓
Java REST Controller
  ↓
Service / Business Logic
  ↓
Repository / DAO
  ↓
Database
```

Error Handling

- Show a loading indicator while requests are running.
- Handle connection failures without crashing the application.
- Display meaningful messages for HTTP errors.
- Avoid displaying raw server exceptions to the user.
- Refresh the UI only after a successful operation.

21.Database and Data Models

The platform uses a normalized relational database design to store the main entities and their relationships. The core tables are users, skills, user_skills, learning_goals, study_matches, and connection_requests.

Table	Purpose	Important Relationship
users	Stores user account and profile information.	Referenced by most user-related tables.
skills	Stores the available skill entities.	Connected to users through user_skills.
user_skills	Associates users with skills.	Many-to-many relationship between users and skills.
learning_goals	Stores what users want to learn.	References the user who owns the goal.
study_matches	Stores or represents compatible study-partner matches.	Connects suitable users based on learning information.
connection_requests	Stores requests between users.	References sender and receiver users.

Conceptual Relationships

```
users 1 ──────< learning_goals

users >──────< user_skills >────── skills

users 1 ──────< study_matches >────── 1 users

users 1 ──────< connection_requests >────── 1 users
                sender / receiver
```

Dart Model Example

```
class LearningGoal {
  final int id;
  final int userId;
```

```
  final String goal;

  LearningGoal({
    required this.id,
    required this.userId,
    required this.goal,
  });

  factory LearningGoal.fromJson(Map<String, dynamic> json) {
    return LearningGoal(
      id: json['id'],
      userId: json['userId'],
      goal: json['goal'],
    );
  }
}
```

22.Animations and User Experience

Animations are used only where they improve feedback or make transitions easier to understand. The project does not require quiz-specific result animations or leaderboard effects.

Suitable Animations

- Fade-in for dashboard sections.
- Subtle card entrance animations for recommended matches.
- Progress indicators during authentication and API calls.
- Animated transitions when opening or closing sections.
- Button feedback after sending a connection request.

Example

```
AnimatedOpacity(
  opacity: isVisible ? 1.0 : 0.0,
  duration: const Duration(milliseconds: 300),
  child: const ProfileHeader(),
)
```

Animations should remain lightweight so that they do not interfere with navigation, scrolling, or API-driven updates.

23.Testing and Debugging

Testing verifies that each module performs the expected operation and that different modules work together. The Skill Sharing Community Platform requires UI, functional, API, and integration testing.

Unit Testing

- Test validators for email, password, skill, and goal input.
- Test model JSON parsing.
- Test helper functions and matching-related calculations where applicable.

Widget Testing

- Verify that the Login screen displays its fields and button.
- Verify that SkillCard displays the expected skill name.
- Verify that MatchCard triggers its connection action.
- Verify that validation messages appear for invalid input.

Integration Testing

- Registration → Login → Dashboard.
- Add Skill → Save → Skill appears in Profile.
- Add Learning Goal → Save → Goal appears in Profile.
- Load Matches → Select Match → Send Connection Request.
- Receive Request → Accept/Reject → Updated status is displayed.

Manual Testing

Manual testing is performed on a physical Android device or configured emulator to check navigation, scrolling, keyboard behavior, API responses, loading states, and visual consistency.

Debugging Tools

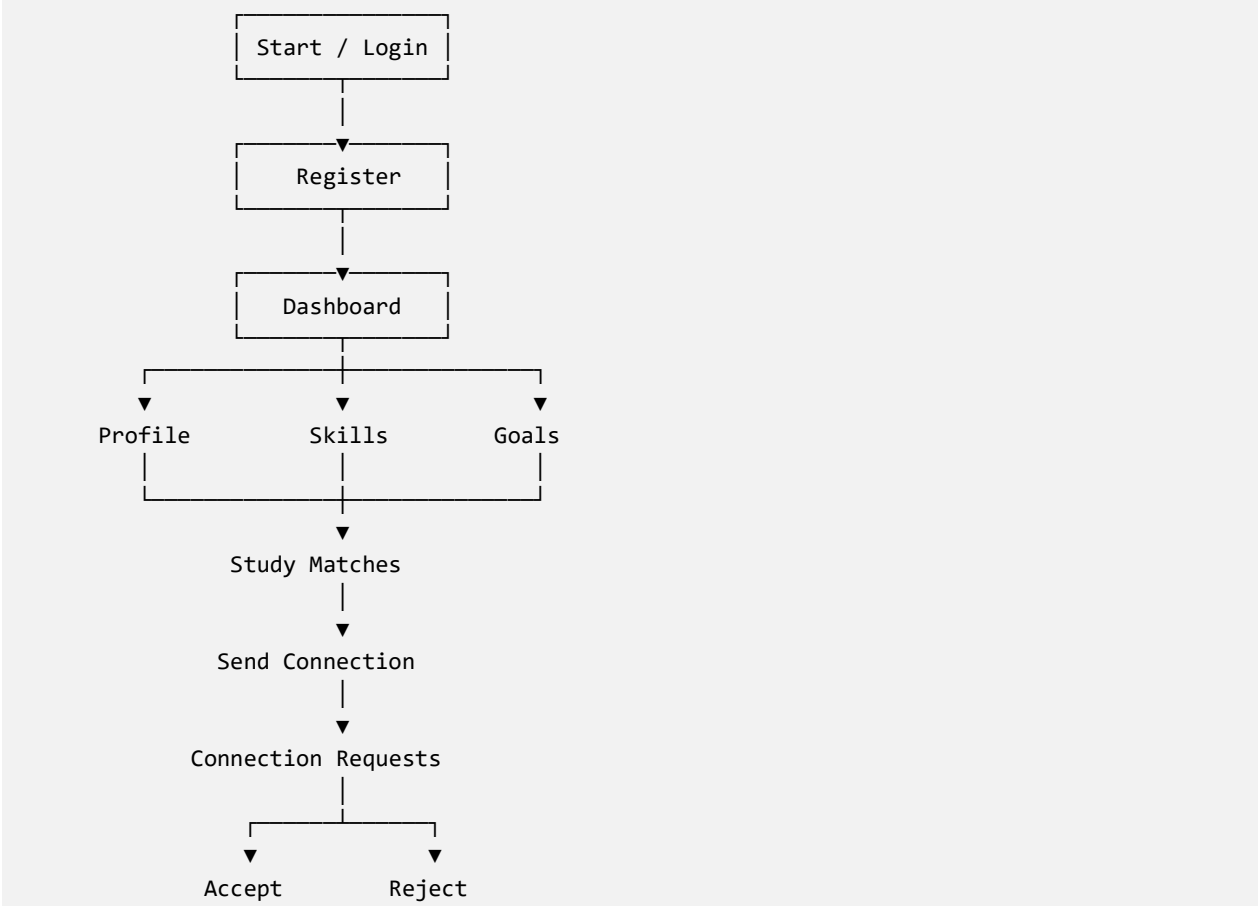
- Flutter DevTools.
- VS Code debugger.
- Console/log output.
- try/catch error handling.
- API testing tools such as Postman when required for backend verification.

Example Error Handling

```
try {
  final matches = await matchService.getMatches();
  setState(() {
    matchList = matches;
  });
} catch (e) {
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text("Unable to load matches"),
    ),
  );
}
```

24.Application Workflow

Complete Screen Workflow



Dashboard Responsibilities

- Provide a central starting point after authentication.
- Show quick access to profile, skills, goals, matches, and requests.
- Display appropriate loading and error states.
- Avoid duplicating the complete logic of child modules.

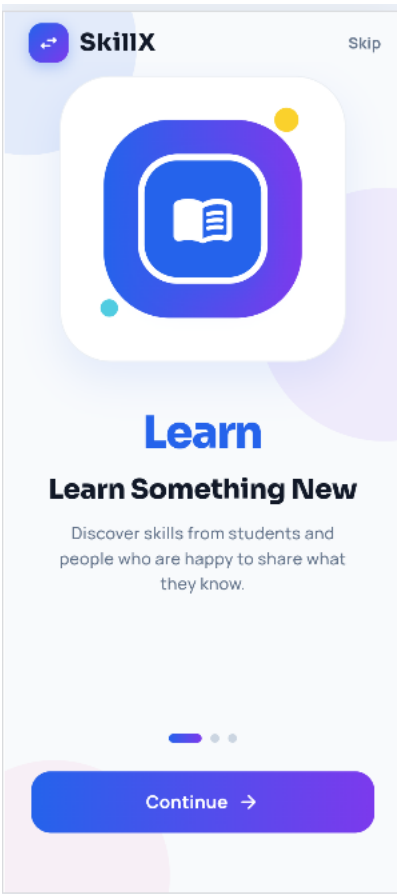
Data Flow

```
User Action
    ↓
Flutter Widget
```

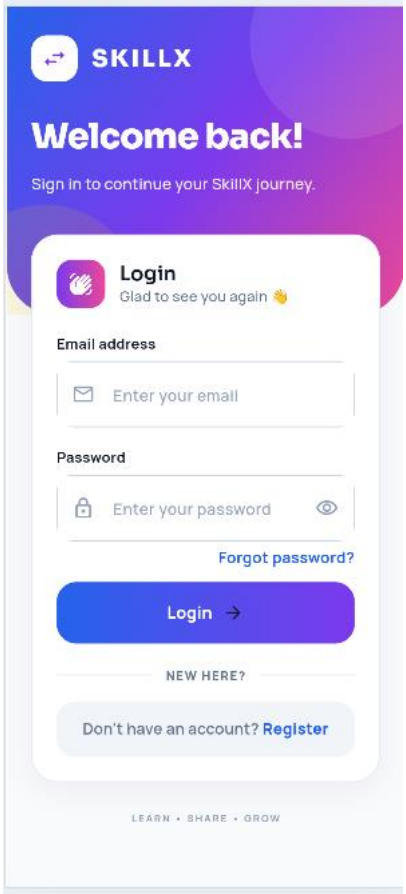
```
    ↓
Provider / setState
    ↓
Service Method
    ↓
REST API
    ↓
Java Backend
    ↓
Database
    ↓
JSON Response
    ↓
Dart Model
    ↓
Updated UI
```



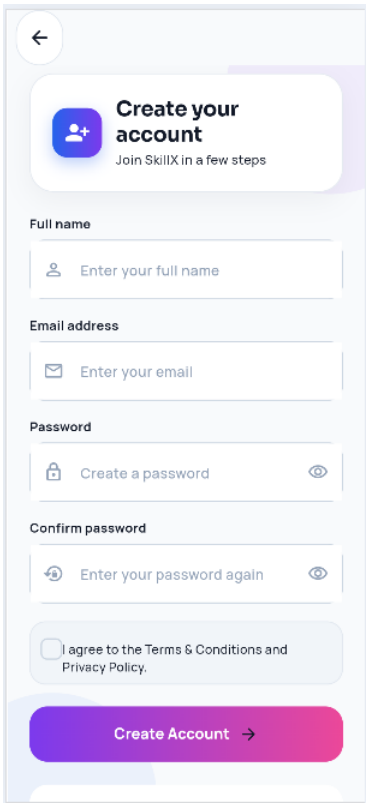
Splash Screen



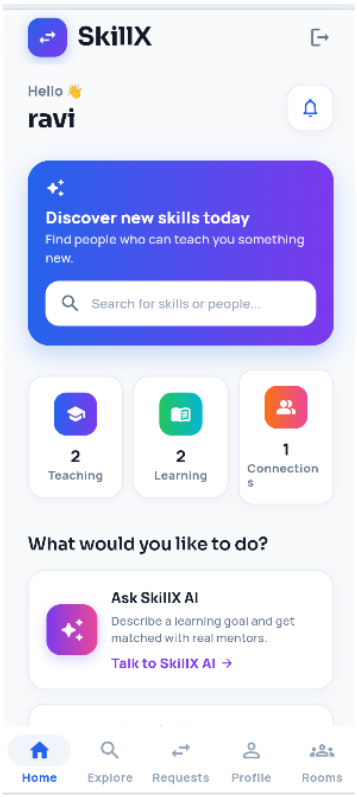
Onboarding Screen



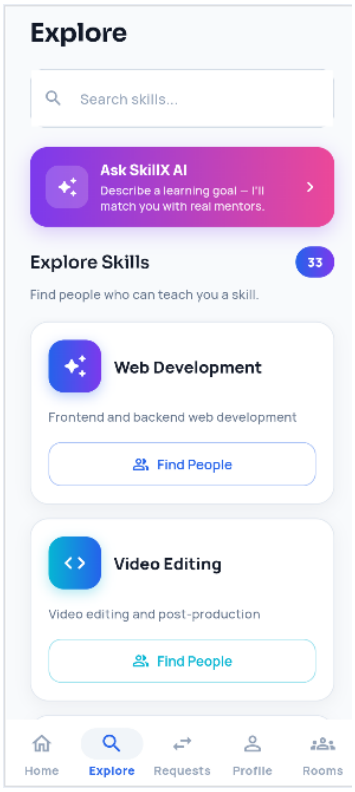
Login Screen



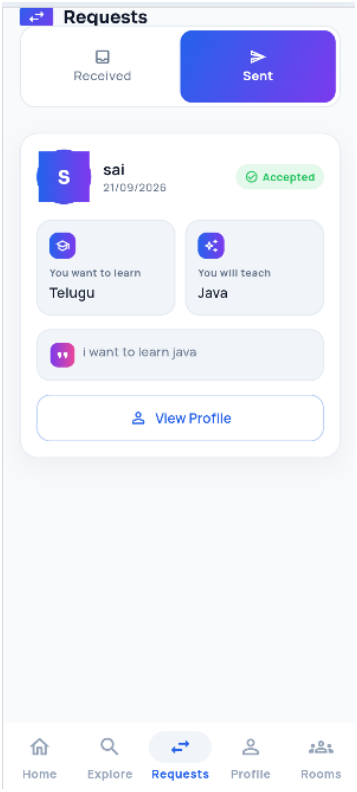
Register Screen



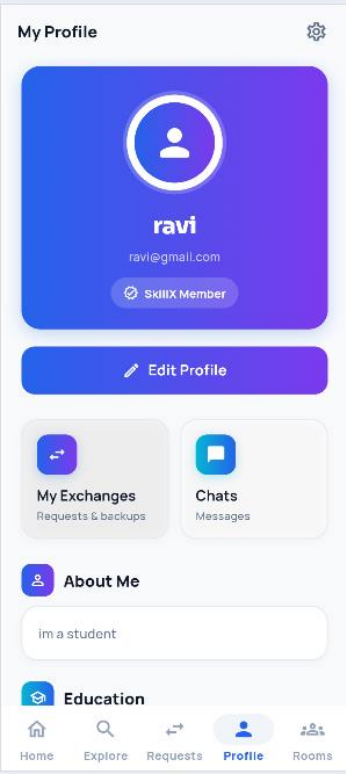
Home Screen



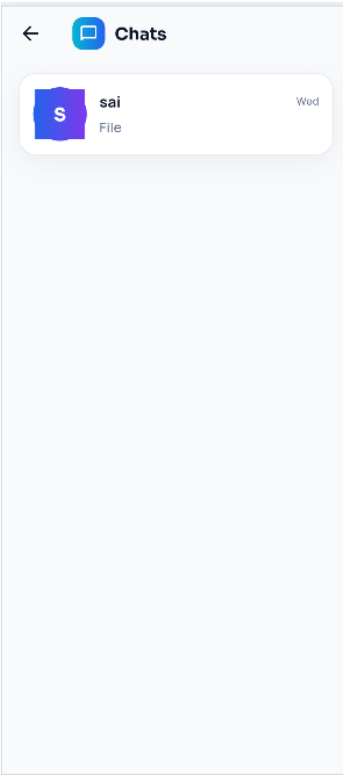
Explore Screen



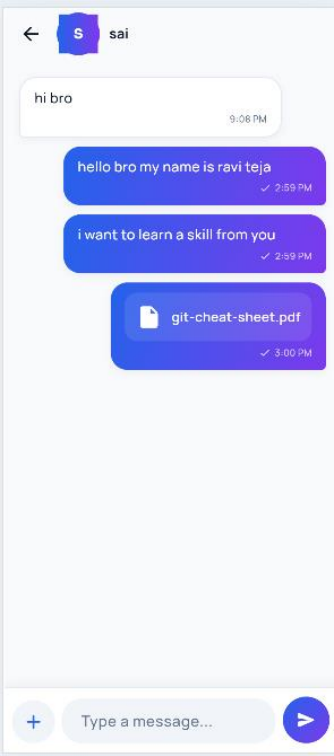
Requests Screen



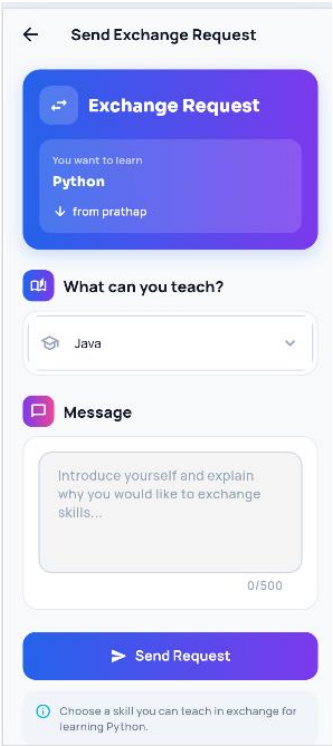
Profile Screen



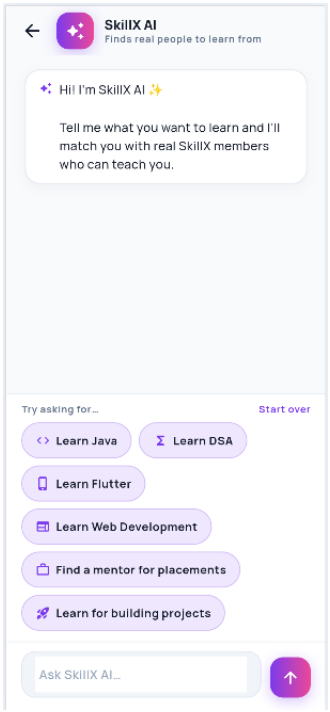
Chat List Screen



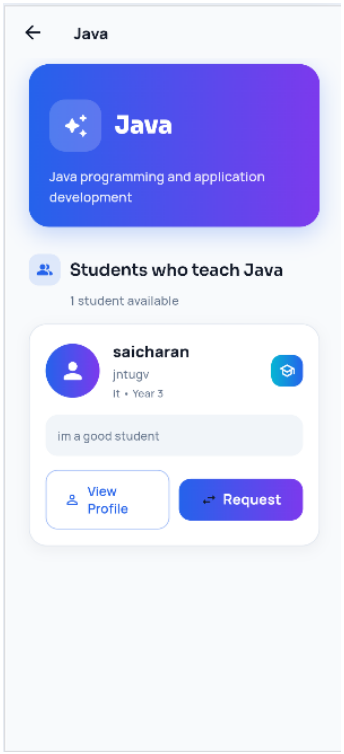
Chat Screen



Send Request Screen



SkillX AI Agent Screen



Skill Card Screen

25.Security and Data Validation

Because the platform stores account and user-generated information, security must be considered at both the Flutter and backend levels.

Application-Side Practices

- Do not hard-code passwords or private credentials in the Flutter source.
- Validate all user input before sending requests.
- Use HTTPS when communicating with a production backend.
- Store authentication/session information using an appropriate secure mechanism.
- Do not display sensitive server errors directly to users.
- Prevent navigation to authenticated screens when the user is not authenticated.

Backend-Side Practices

- Validate all incoming API parameters.
- Authenticate protected API requests.
- Authorize operations so users can modify only permitted resources.
- Use parameterized database queries or a safe data-access framework.
- Enforce database constraints for required relationships.
- Return appropriate HTTP status codes.

Why Both Layers Matter

Flutter validation improves the user experience, but the backend is the final authority for security and data integrity. A malicious or modified client can bypass UI validation, so server-side validation is essential.

26.Conclusion

The Skill Sharing Community Platform demonstrates how Flutter and Dart can be used to build a practical mobile application for peer-to-peer learning. The project goes beyond basic UI development by combining authentication, profile management, skill management, learning goals, study-partner matching, connection requests, state management, responsive design, and REST API communication.

The application architecture separates screens, reusable widgets, data models, and service classes. This structure makes the project easier to understand, test, maintain, and extend. The Flutter front end communicates with the Java backend through REST APIs, while the relational database stores the core entities and relationships.

The project also demonstrates important software-development practices such as input validation, asynchronous programming, error handling, responsive layouts, reusable components, and systematic testing. These concepts provide a strong foundation for developing larger full-stack mobile applications.

Final Project Summary

Area	Implemented / Required Focus
Frontend	Flutter and Dart
Authentication	Registration, login, logout
Profile	View and edit user information
Skills	Skill discovery and user-skill association
Learning Goals	Create and manage learning goals
Matching	Study-partner discovery using skills and goals
Connections	Connection request workflow
Backend	Java REST API
Database	Relational database with normalized core tables
State	Local state plus shared state management where required

Testing	Unit, widget, integration, and manual testing

This documentation is focused exclusively on the Skill Sharing Community Platform. The original sample-application material that was unrelated to this project has been removed, and the documentation has been reorganized around the platform's actual authentication, profile, skills, learning goals, study matching, connection requests, Flutter UI, state management, REST API integration, database, testing, and security requirements.